

Métricas de qualidade para orientação a objetos

Atividade apresentada como parte do processo de avaliação da disciplina de Engenharia de Software I do curso de Bacharelado em Sistemas de Informação da Faculdade de Ciências da UNESP de Bauru.

Integrantes do grupo:

RA	Nome (completo)
241025265	Igor dos Reis Gomes

Código 1 (Celso):

Os seguintes resultados foram obtidos:

	Package	TLOC	NOC
0	helpers	462.0	3

Tabela 1 - Métricas de Pacote

Nesta tabela, nota-se que o pacote helpers possui um total de 462 linhas de código, que são distribuídas em um total de 3 classes. Portanto, percebe-se que essas classes são bem extensas.

	Class	NF	NM	DIT	WMC	LCOM_HS	CF
0	Funcoes	2	34	1	41.0	1.015152	0.0
1	JSWaiter	3	8	1	14.0	0.666667	0.0
2	WaiterForAngular	3	5	1	8.0	0.750000	0.0

Tabela 2 - Métricas de Classe

Já a tabela 2, percebe-se que a classe Funcoes se destaca de forma negativa, já que ela possui um total de 34 métodos, um número bem superior em relação às outras classes. Refletindo assim no WMC com valor de 41, o mais alto entre as 3 classes, indicando que é a classe mais complexa do pacote

	Class	Method	VG	Fout	Fin
0	Funcoes	public static Long regexNum(WebDriver driver, ...	1.0	0.0	0.0
1	Funcoes	public static String dataAtual()	1.0	0.0	0.0
2	Funcoes	public static String geraCel()	2.0	0.0	0.0
3	Funcoes	public static String geraCep()	2.0	0.0	0.0
4	Funcoes	public static String geraData()	1.0	0.0	0.0

Tabela 3 - Métricas de Método

A tabela 3 reforça ainda mais a complexidade da classe Funcoes, já que os métodos geraCel e geraCep se destacam dentre todas as classes, possuindo VG = 2, possuindo uma complexidade maior que as outras classes.

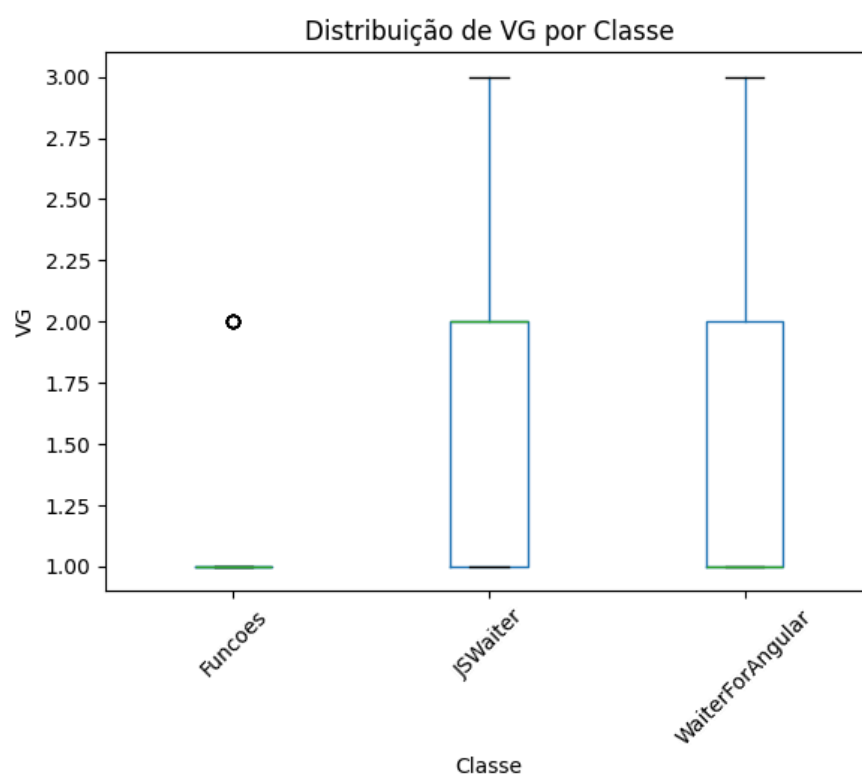


Figura 1 - Distribuição de VG por classe

No boxplot da figura 1, na classe Funcoes pode-se verificar que, apesar de muito grande, a grande maioria dos métodos presentes nessa classe são pouco complexos, tendo valor de VG = 1. Além disso, possui apenas um outlier de valor 2.0. Portanto,

nota-se que a complexidade dessa classe não vem a partir de lógicas complexas dentro dessa classe, mas sim a enorme quantidade de métodos simples.

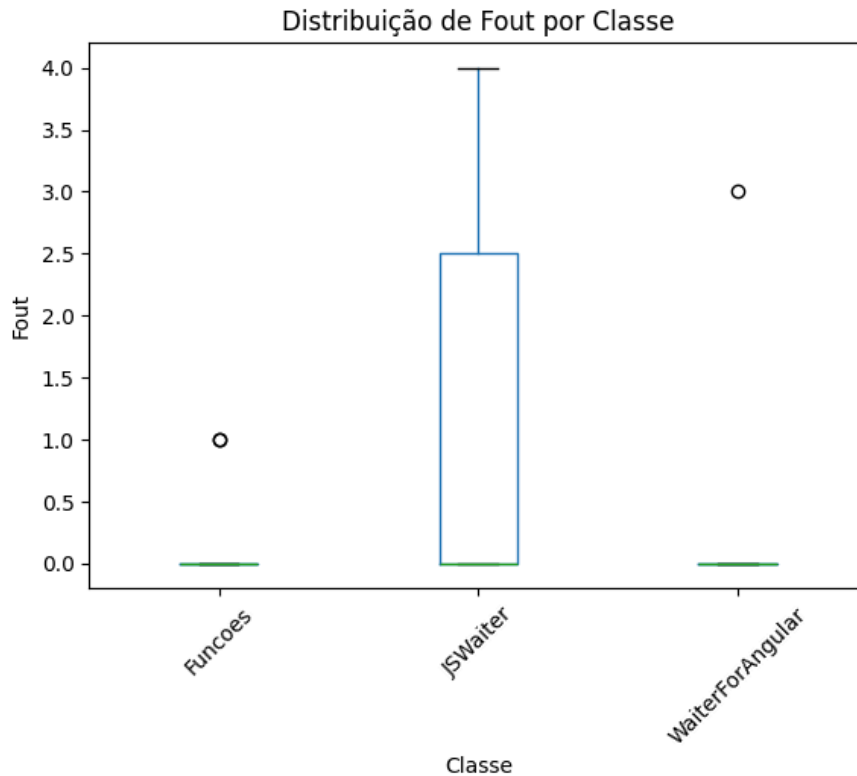


Figura 2 - Distribuição de Fout

O boxplot presente na figura 2 mostra a distribuição de quantas chamadas “para fora” os métodos de cada classe fazem. O grande destaque é a classe JSWaiter, classe que os métodos mais chamam outras partes do sistema. Portanto, verifica-se que os métodos presentes na classe JSWaiter são mais acoplados e dependem de outras classes para funcionar.

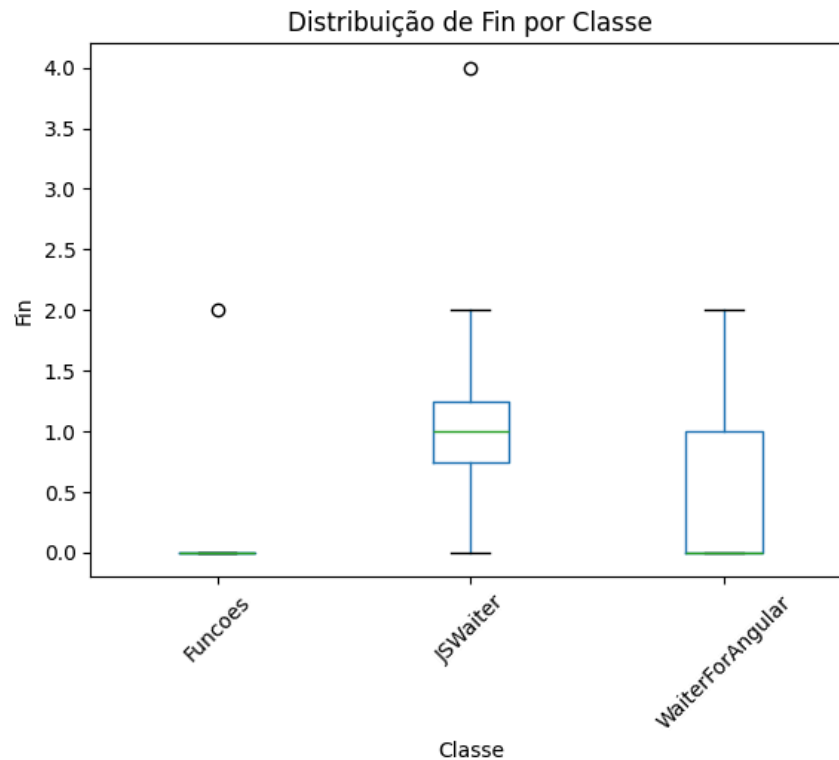


Figura 3 - Distribuição de Fin

Por fim, esse boxplot mostra a distribuição de quantas vezes os métodos de cada classe são chamados por outros métodos. O componente que mais se destaca é a classe Funcoes, que juntamente com a Tabela 3, pode-se destacar que a grande maioria dos 34 métodos existentes na classe Funcoes não está sendo usada por nenhuma outra classe, apenas um único método é chamado 2 vezes (outlier). Tendo assim um problema grande de código que não está sendo utilizado.

Código 2 (Capítulo 34):

Foram obtidos os seguintes resultados:

	Package	TLOC	NOC
0	default	359.0	12

Tabela 1 - Métricas de Pacote

A tabela 1 descreve que todo o sistema está dentro do pacote default, com um total de 359 linhas de código espalhadas por 12 classes diferentes. Destaca-se então, que esse projeto é composto por muitas classes pequenas e com responsabilidades bem definidas.

	Class	NF	NM	DIT	WMC	LCOM_HS	CF
0	ATM	11	6	1	17.0	0.636364	0.0
1	ATMCaseStudy	0	1	1	1.0	0.000000	0.0
2	Account	4	7	1	8.0	0.708333	0.0
3	BalanceInquiry	0	2	1	2.0	0.000000	0.0
4	BankDatabase	1	7	1	9.0	0.833333	0.0
5	CashDispenser	2	3	1	4.0	0.500000	0.0
6	Deposit	4	3	1	6.0	0.625000	0.0
7	DepositSlot	0	1	1	1.0	0.000000	0.0
8	Keypad	1	2	1	2.0	0.000000	0.0
9	Screen	0	3	1	3.0	0.000000	0.0
10	Transaction	3	5	1	5.0	0.750000	0.0
11	Withdrawal	4	3	1	14.0	0.625000	0.0

Tabela 2 - Métricas de Classe

A tabela acima faz um detalhamento maior de cada uma das 12 classes presentes nesse projeto. Como destaque, temos a classe ATM, que claramente é a classe central e

mais complexa, já que possui um número maior de atributos (11), o que indica que essa classe conhece e coordena outros componentes. Além disso, possui o WWC mais alto dentre todas as classes (17.0), o que significa que a lógica principal e mais complexa do projeto está na classe ATM.

	Class	Method	VG	Fout	Fin
0	ATM	private Transaction createTransaction(int type)	4.0	0.0	1.0
1	ATM	private int displayMainMenu()	1.0	0.0	1.0
2	ATM	private void authenticateUser()	2.0	0.0	1.0
3	ATM	private void performTransactions()	6.0	2.0	1.0
4	ATM	public ATM ATM()	1.0	0.0	0.0

Tabela 3 - Métricas de Método

Já a tabela 3 mostra um recorte de todos os métodos presentes dentro da classe ATM. O método que se destaca é o performTransactions(), possuindo maior complexidade (VG=6.0) e é o único método dentre os listados que possui Fout, significando que esse método chama outros métodos para delegar tarefas.

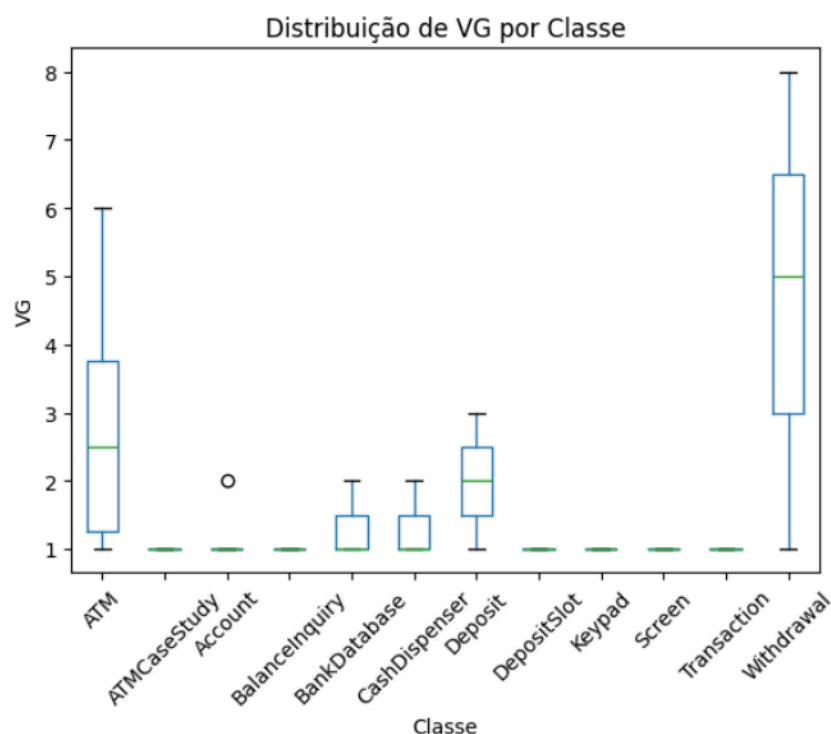


Figura 1 - Distribuição de VG por classe

Na figura 1, destaca-se a classe Withdrawal, por ter os métodos mais complexos do sistema. Enquanto a maioria das outras classes possuem apenas métodos simples ($VG = 1$). Essa característica faz sentido, uma vez que uma operação de classe é inevitavelmente complexa. É necessário verificar o saldo da conta, verificar o saldo disponível no caixa, fazer com que o caixa libere as notas e debitar o valor na conta.

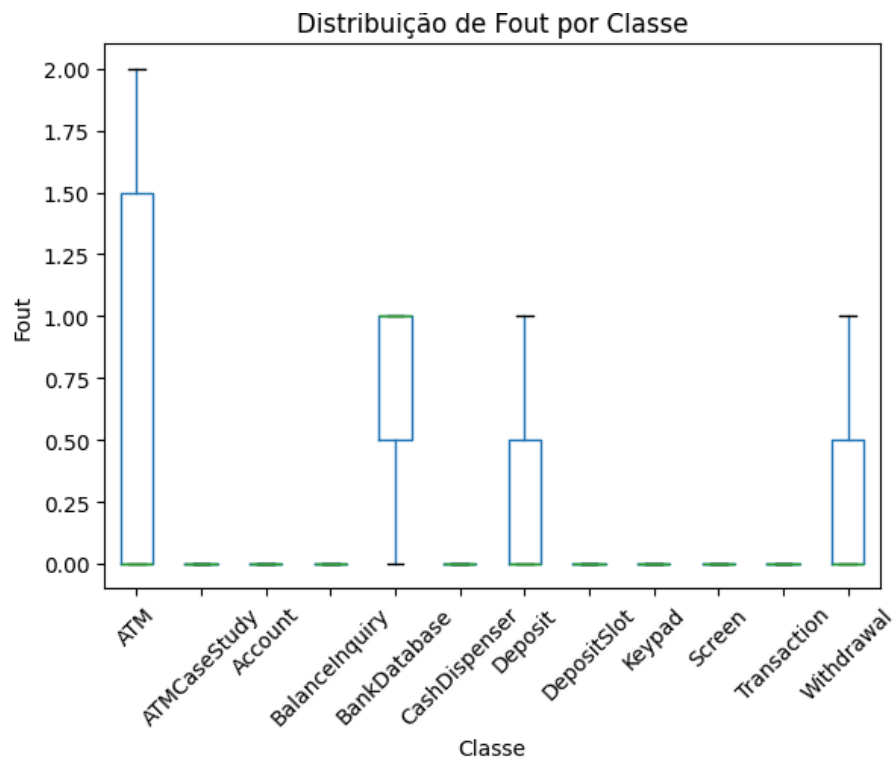


Figura 2 - Distribuição de Fout

Na figura 2, o boxplot tem como item em destaque a classe ATM, confirmando o que foi dito anteriormente da classe ATM ser a classe mais importante e que controla o sistema, já que a grande maioria das outras classes possuem $Fout = 0$, sendo assim, são componentes que são usados e não usam outras classes, comportamento contrário à classe ATM.

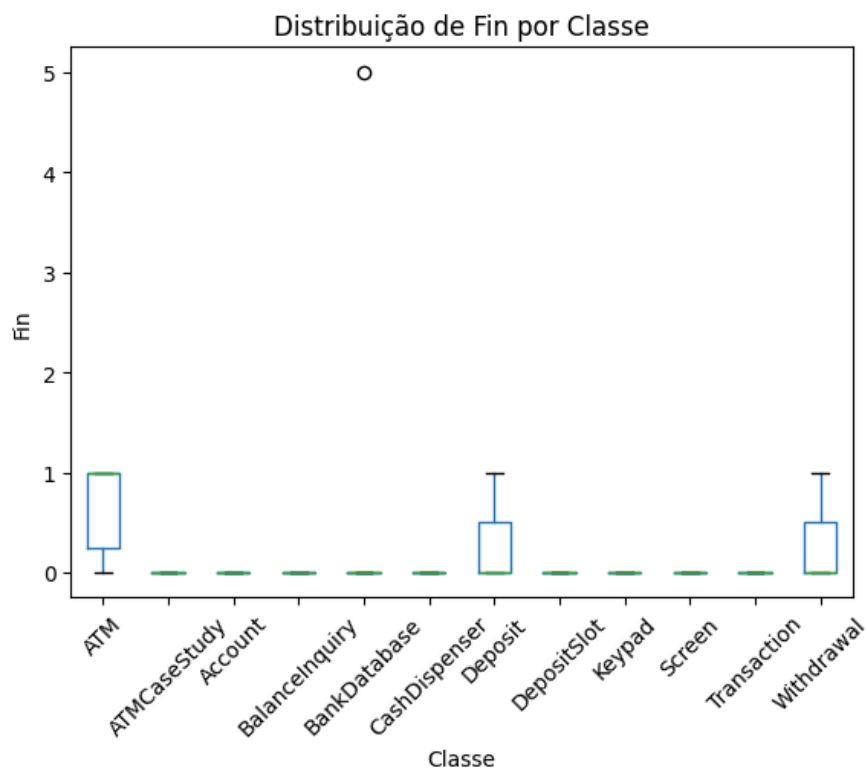


Figura 3 - Distribuição de Fin

Por fim, na figura 3 a classe que mais se destaca é a BalanceInquiry, possuindo um outlier de valor 5. Na tabela 2, mostra que essa classe possui apenas 2 métodos e no gráfico é mostrado que um desses métodos é chamado por 5 locais diferentes no código, um valor muito alto em comparação com todos os outros métodos.